



Service API pour IoT

Étude de cas

Soumis par:

Ramanitranja, Raphael

Introduction

Ce projet présente le développement d'une API légère et robuste conçue pour l'ingestion, la validation en temps réel et l'analyse de données de télémétrie IoT. Cette solution intègre une architecture orientée domaine, un moteur de détection d'anomalies et une stratégie de persistance flexible pour répondre aux exigences des environnements distribués à grande échelle.

Architecture

L'architecture de l'API a été conçue pour être modulaire et prête pour une mise à l'échelle. Elle repose sur le serveur ASGI Uvicorn pour assurer un traitement asynchrone et elle utilise le framework FastAPI. Elle suit une structure orientée domaine afin d'isoler les responsabilités par fonctionnalités d'affaires (tel que la télémétrie) pour faciliter la maintenance future. Le traitement des données suit un pipeline unidirectionnel strict dans lequel les payloads entrant sont d'abord validés à la passerelle par des schémas Pydantic (`telemetry/schemas.py`), puis évalués par un moteur de règles centralisé (`services/anomaly_engine.py`) chargé de détecter les anomalies. Pour la couche de persistance, une approche hybride a été privilégiée avec SQLAlchemy (configurée via `database/database.py` et modélisée dans `database/models.py`). L'utilisation d'une colonne JSON permet de garantir que le système peut incorporer de nouveaux types de capteurs avec leurs propres champs et métadonnées sans nécessiter de lourdes migrations structurelles. Ensuite, l'ensemble est exposé via le routeur (`telemetry/router.py`) et validé par une suite de tests automatisés (`tests/`) propulsée par Pytest.

Protocole de tests

L'environnement de test est bâti sur la librairie pytest. Pour tester les routes API sans nécessiter de requêtes réseau réelles, la librairie TestClient de FastAPI est utilisée, ce qui permet de simuler les appels HTTP de manière rapide et synchrone. La suite de tests est divisée en deux niveaux distincts pour assurer une couverture optimale. Il y a tout d'abord des tests unitaires permettant de valider directement la logique du moteur d'anomalies. Ceux-ci vérifient les cas tels que le traitement d'un payload sain (aucune anomalie), la détection d'une anomalie critique isolée (ex: température à 1,5 C pour la règle A-002), et la détection de multiples anomalies simultanées se retrouvant sur un même payload. Ensuite, il y a les tests d'intégration qui valident le pipeline complet des données allant de l'ingestion à la validation Pydantic et le stockage dans la base de données. Deux cas sont testés, soit une ingestion réussie avec le retour de la variable `saved_id` et la validation du comportement de la route de lecture (GET `/telemetry/anomalies`) suite à l'injection volontaire d'un payload erroné.

Voici les structures JSON utilisées pour simuler les différents scénarios fonctionnels lors des tests d'intégration:

A. Cas nominaux (Aucune anomalie - Succès)

Ce payload représente un appareil fonctionnant dans des conditions optimales.

```
{
  "device_id": "TEST-001",
  "device_type":
    "kepler",
  "building_id":
    "MVTCC",
  "unit": "101",
  "timestamp":
    "2026-07-21T18:00:00Z",
  "values": { "temperature_c": 22.5, "humidity_pct": 45.0, "leak_resistance_ohm": 200000,
    "battery_pct": 90, "signal_quality": -50 }
}

{
  "device_id": "KEP-402",
  "device_type": "kepler",
  "building_id": "EST",
  "unit": "301",
  "timestamp": "2026-07-21T21:15:00Z",
  "values": {
    "temperature_c": 22.4,
    "humidity_pct": 45.2,
    "leak_resistance_ohm": 99999,
    "battery_pct": 88,
    "signal_quality": 4
  }
}

{
  "device_id": "NEW-101",
  "device_type": "newton",
  "building_id": "CENTRE",
  "unit": "Lobby",
  "timestamp": "2026-07-21T21:16:10Z",
  "values": {
    "motion_detected": true,
    "luminosity_lux": 350.5,
    "temperature_c": 21.0
  }
}
```

B. Cas d'anomalie (Déclenchement des règles)

```
{
  "device_id": "KEP-TEMP-01",
  "device_type": "kepler",
  "building_id": "NORD",
  "unit": "101",
  "timestamp": "2026-07-21T22:10:00Z",
  "values": {
    "temperature_c": 5.0,
    "humidity_pct": 40.0,
    "leak_resistance_ohm": 150000,
    "battery_pct": 80,
    "signal_quality": -70
  }
}
```

```
{
  "device_id": "KEP-TEST-01",
  "device_type": "kepler",
  "building_id": "EST",
  "unit": "101",
  "timestamp": "2026-07-21T22:00:00Z",
  "values": {
    "temperature_c": 21.0,
    "humidity_pct": 50.0,
    "leak_resistance_ohm": 500,
    "battery_pct": 10,
    "signal_quality": -90
  }
}
```

```
{
  "device_id": "GAL-HB-01",
  "device_type": "galileo",
  "building_id": "OUEST",
  "unit": "502",
  "timestamp": "2026-07-21T22:18:00Z",
  "values": {
    "heartbeat": "WARNING",
    "cpu_usage_pct": 15.0,
    "disk_free_gb": 100.0,
    "service_status": "healthy"
  }
}
```

C. Cas de validation refusée (Rejet par Pydantic)

```
{
  "device_id": "TEST-003",
  "device_type": "capteur_inconnu",
  "building_id": "MVTCC",
  "unit": "101",
  "timestamp": "2026-07-21T18:10:00Z",
  "values": { "temperature_c": 20.0 }
}

{
  "device_id": "GAL-999",
  "device_type": "galileo",
  "building_id": "OUEST",
  "unit": "502",
  "timestamp": "2026-07-21T21:22:00Z",
  "values": {
    "heartbeat": "OK",
    "cpu_usage_pct": 12.0,
    "disk_free_gb": 100,
    "service_status": "healthy",
    "setpoint_c": 22.0
  }
}

{
  "device_id": "THR-888",
  "device_type": "thermostat",
  "building_id": "NORD",
  "unit": "99",
  "timestamp": "2026-07-21T21:25:00Z",
  "values": {
    "setpoint_c": 20.0,
    "room_temperature_c": 19.0,
    "heating_active": "active"
  }
}
```

Avant de tester les payloads, il faut s'assurer d'être à la racine du projet dans le terminal et de démarrer le serveur de développement Uvicorn:

py -m uvicorn main:app --reload

une fois le serveur démarré, on peut explorer et tester manuellement les routes de l'API en ouvrant le navigateur à l'adresse suivante:

<http://127.0.0.1:8000/docs>

Cette interface interactive permet de visualiser l'ensemble des points de terminaison (POST/telemetry et GET/telemetry/anomalies), de consulter les schémas de validation Pydantic et d'exécuter des requêtes directement en y insérant les exemples de payload fournis. Il faut repérer la route POST correspondante (POST / telemetry) et cliquer dessus pour dérouler le bloc. Ensuite, on peut cliquer sur le bouton "Try it out" situé en haut à droite du bloc afin de rendre la zone de texte du payload modifiable et on pourra ensuite copier et coller les payloads. Cliquer finalement sur Execute. Si le payload est valide, on recevra un code de status 200 OK avec la réponse du serveur et si des anomalies sont détectées.

Ensuite, pour consulter les anomalies on repère la route GET correspondante (GET /telemetry/anomalies) et cliquer dessus pour dérouler le bloc. On peut cliquer sur le bouton "Try it out" situé en haut à droite du bloc et configurer les filtres au besoin ou les laisser vide pour voir l'entièreté des anomalies. Après avoir cliqué sur le bouton "Execute", on peut observer la réponse sous forme de liste au format JSON de toutes les anomalies détectées et enregistrées dans la base de données suite au envois précédents.

Pour valider la logique du moteur d'anomalies et les routes d'intégration via Pytest, exécuter la commande suivante dans le terminal:

```
py -m pip install pytest httpx
```

```
py -m pytest
```

Cloud Mapping

Bien que cette preuve de concept soit développée localement avec SQLite, l'architecture a été pensée pour être migrée vers une infrastructure Cloud native. Voici le mapping suggéré sur Microsoft Azure pour un déploiement à grande échelle:

1. Point d'entrée et Ingestion (API Gateway & Compute)

Actuel: Serveur Uvicorn local

Cloud: Azure API Management pour gérer et sécuriser le trafic HTTP, redirigeant vers l'application FastAPI conteneurisée et hébergée sur Azure Container Apps. Ce service permettrait d'ajuster dynamiquement le nombre d'instances pour absorber les pics de transmission des capteurs

2. Base de données et Stockage

Actuel: SQLite (Base de données relationnelle locale avec colonne JSON)

Cloud: Azure Database for PostgreSQL PostgreSQL gère nativement le format JSONB qui coïncide parfaitement avec notre choix architectural de stocker les valeurs dynamiques de télémétrie et les métadonnées d'anomalie dans une colonne flexible, sans figer le schéma

3. Moteur d'anomalies

Actuel: Traitement synchrone dans la route FastAPI

Cloud: Pour garantir une surveillance robuste des systèmes distribués et traiter de très hauts volumes, l'API pousserait les payloads bruts dans Azure Event Hubs. des fonctions Azure consommeraient ensuite ces événements pour évaluer les règles de manière asynchrone permettant de gérer des intervalles de temps complexes sans bloquer la réponse de l'API.

Réflexion, Limites et Prochaines Étapes

Bien que cette preuve de concept réponde efficacement aux exigences d'ingestion, de validation et de détection d'anomalies, sa conception actuelle présente certaines limites liées à un traitement purement synchrone et sans état. Un élément notable est l'implémentation de la règle A-006 concernant le seuil de flux, où l'évaluation d'une baisse anormale nécessite une vision globale. Dans un cas isolé, il est difficile de calculer des moyennes mobiles ou des variations de débit dans le temps sans historique persistant. La prochaine étape pourrait être par exemple de découpler l'ingestion du traitement en introduisant un courtier de messages pour absorber les pics de trafic de manière asynchrone.